

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I V.SURYA SATVIKA (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: v.surya satvika

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A

/ \

B C

/\ /\

D E F G

Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,

A

/ \

B C

/\ /\

D E F G

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Qusetion 1:-

Aim: To implement the Breadth-First Search (BFS) algorithm in Python to traverse a graph level by level from a given starting node and display the traversal order along with the queue contents at each iteration.

Program:

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def bfs(graph, start):
    visited = []
    queue = deque([start])
    iteration = 1
    print("{:<10} {:<25} {}".format("Iteration", "Queue", "Visited Node"))
    print("-" * 50)
    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.append(node)
            for neighbour in graph[node]:
                if neighbour not in visited and neighbour not in queue:
                    queue.append(neighbour)
            print("{:<10} {:<25} {}".format(
                iteration,
                ", ".join(queue) if queue else "Empty",
                node
            ))
    ))
```

```

        iteration += 1
    print("\nBFS Traversal:", " -> ".join(visited))
start_node = "A"
bfs(graph, start_node)

```

Output:

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

BFS Traversal: A -> B -> C -> D -> E -> F -> G

Question 2:-

Aim: To implement the Depth-First Search (DFS) algorithm in Python to traverse a graph from a given starting node and display the traversal order using a stack.

Program:

```

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}
visited = []
def dfs(node):
    if node not in visited:

```

```

    print(node, end=" ")
    visited.append(node)
    for neighbour in graph[node]:
        dfs(neighbour)
print("DFS Traversal:")
dfs('A')

```

Output:

```

DFS Traversal:
A B D E C F G
|

```

Dry Run:-

Iteration	Stack	Visited Node
1	A	A
2	B, C	B
3	D, E, C	D
4	E, C	E
5	C	C
6	F, G	F
7	G	G

Question 3:-

Aim: To implement the Breadth-First Search (BFS) algorithm in Python to solve the 8-puzzle problem by finding the shortest path from the initial state to the goal state.

Program:-

```

from collections import deque
goal = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))

```

```

start = ((1, 3, 6),
         (5, 0, 2),
         (4, 7, 8))
moves = [(-1,0),(1,0),(0,-1),(0,1)]
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            temp = [list(row) for row in state]
            temp[x][y], temp[nx][ny] = temp[nx][ny], temp[x][y]
            neighbors.append(tuple(tuple(row) for row in temp))
    return neighbors
def bfs(start, goal):
    queue = deque([(start, [])])
    visited = set()
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state not in visited:
            visited.add(state)
            for neighbor in get_neighbors(state):
                if neighbor not in visited:
                    queue.append((neighbor, path + [state]))
    return None
solution = bfs(start, goal)
if solution:

```



```

print("Solution Path:\n")
for step in solution:
    for row in step:
        print(row)
    print()
else:
    print("No Solution Found")

```

Output:-

```

=====
Solution Path:

(1, 3, 6)
(5, 0, 2)
(4, 7, 8)

(1, 3, 6)
(5, 2, 0)
(4, 7, 8)

(1, 3, 0)
(5, 2, 6)
(4, 7, 8)

(1, 0, 3)
(5, 2, 6)
(4, 7, 8)

(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

```

Answers:-

- 1.The front node (state) in the queue is expanded during each iteration.
- 2.The total number of generated states depends on the BFS execution and cannot be determined without running the algorithm.
- 3.The complete solution path is obtained by executing the BFS algorithm and tracing the sequence of states from the initial state to the goal state.
- 4.BFS explores states level by level, and since every move has the same cost, the first time the goal state is reached it is guaranteed to be the shortest path (minimum number of moves).